

Composing O-RAN, TM Forum, IEEE, DMTF, and Kubernetes-Native Contracts Without Inventing Parallel Standards: A Bridge-Based Architecture for Closed-Loop Cloud RAN Remediation

David Kypuros

Red Hat

dkypuros@redhat.com

31 May 2026

Abstract

A closed-loop remediation harness for O-RAN Cloud RAN infrastructure must touch contracts from at least five standards bodies (O-RAN Alliance, TM Forum, IEEE, DMTF, CNCF / Kubernetes SIGs) to be operationally complete. The dominant pattern in published agentic-RAN work either elevates one body as the authoritative frame and maps the others into it loosely, or introduces a parallel specification to bridge them. This paper describes an alternative: each standards body retains authority over its own domain, and the harness composes them at well-defined seam points. We identify seven named bridges, document each with a file-level citation header in the source repository, and enforce bidirectional conformance through a 10-check verify gate any reviewer can run after cloning. We claim the composition approach makes the resulting architecture more durable across spec revisions and politically simpler to land in a multivendor ecosystem. v0 limits are explicit: several bridges are present at the schema and stub layer, not exercised against live spec implementations.

1 Introduction

Cloud RAN Day-2 operations sit at the intersection of several mature but historically separate standards bodies. The O-RAN Alliance defines the SMO-to-O-Cloud interface family [22–24] and the broader architecture [17]. TM Forum defines the event-management and intent-management API envelopes [35, 36] along with the SID information model [29]. IEEE and ITU-T define the precision timing protocol that gates Cloud RAN service availability [5, 6]. DMTF defines the out-of-band server management interface [4] that BMCs implement. CNCF and Kubernetes SIGs maintain the CloudEvents envelope [3] and the CRD-shaped delivery contracts (MachineConfig [27], KMM Module [8], Metal3 [14, 15]) that actually carry the changes onto a worker node.

An agentic remediation harness has to compose these contracts to produce one operationally complete loop. Two common approaches in prior work do this poorly. The first elevates a single body (often the SMO frame, or sometimes a pure Kubernetes frame) and maps others into it loosely, which produces translation rot and am-

biguous ownership during real fault investigations. The second invents a parallel specification, which produces a third standards body the user community is asked to adopt; this is politically expensive and operationally brittle when the underlying specs revise.

This paper documents a third approach, exercised in the open-source O-RAN Agent Harness referenced by the prior multivendor diagnostic publication [7]. Each body retains authority over its own domain. The harness defines named bridge points where it composes contracts from one body with contracts from another, and each bridge has a file-level citation in the repository. A 10-check verify gate validates that the bridge metadata is internally consistent.

The remainder of the paper is organized as follows. Section 2 surveys the five standards bodies and their domain authority. Section 3 documents seven named bridges in detail. Section 4 describes the conformance auditing mechanism. Section 5 compares this approach to ONAP, Nephio, and OSM. Section 6 concludes.

2 The Standards Stack

O-RAN Alliance. The O-RAN Architecture Description [17] and the Decoupled SMO Architecture technical report [19] define the macro layout. The O2 family [22–24] governs the SMO-to-O-Cloud interfaces. The management-plane family covers O1 [21] (SMO to managed elements), A1 [16] (Non-RT RIC to Near-RT RIC), R1 [25] (SMO-internal service exposure), and E2 [20] (Near-RT RIC to E2 nodes). The harness targets O2 IMS exclusively for infrastructure remediation; other O-RAN interfaces are referenced as scoping boundaries, not consumed.

TM Forum. TMF688 [35] is the event-management envelope used as the audit sink. TMF921 [36] is the intent-management envelope used for the dual-route notification path. TMF634 [33] (resource inventory) and TMF642 [34] (alarm management) are referenced as background context. The SID information framework [29] grounds the conceptual model. Implementation guides for AIOps [31], ODA functional architecture [30], and intent management [32] inform the design without being directly consumed.

IEEE and ITU-T. IEEE 1588-2019 [5] defines the Precision Time Protocol state machine (SYNCHRONIZED, HOLDOVER, FREE_RUNNING, UNCALIBRATED transitions) and the PTP Hardware Clock semantics. The ITU-T G.8275.1 telecom profile [6] specifies the full-timing-support deployment posture that Cloud RAN uses.

DMTF. The Redfish specification [4] governs out-of-band server management. The harness does not consume Redfish directly; Metal3 BMO does, on behalf of the harness, when a firmware push targets a host BMC.

CNCF and Kubernetes SIGs. CloudEvents 1.0 [3] is the vendor-neutral event envelope. Three CRD-shaped operators carry the delivery: Machine Config Operator [27], KMM [8], and Metal3 BMO [14]. The MCP specification [2] and its Python SDK [1] along with the FastMCP framework [13] provide the internal agent-to-tool scaffolding.

3 The Composition Bridges

Each bridge below names the from-spec, the to-spec, the seam artifact in the harness, the file anchor where the bridge is implemented, and the bibliography references involved. Figure 1 shows the seven bridges in one view. A reviewer can confirm any single bridge by reading the named files.

3.1 Bridge 1: IEEE 1588 / G.8275.1 to O-RAN WG6 Cloud Notifications

PTP state transitions defined by IEEE 1588 [5] under the G.8275.1 profile [6] are the raw signal. The linuxptp daemons [12] expose these states. The cloud-event-proxy sidecar [28] translates them into O-RAN.WG6 Cloud Notifications [18] carried over the CNCF CloudEvents 1.0 envelope [3]. The harness consumes the resulting envelope as `FaultPayload`. The bridge chains four standards bodies in one signal path: IEEE / ITU-T timing domain to CNCF event envelope to O-RAN notification contract to the harness’s inbound schema.

3.2 Bridge 2: O-RAN WG6 O-Cloud resource model to harness taxonomy

The harness defines a 20-entry taxonomy of fault classes in `harness/taxonomy.yaml`. The file opens with an explicit grounding disclaimer: the 20 subtypes are interpretive mappings within the three normative O-Cloud resource classes from O2 General Aspects and Principles Figure 3.4-1 [23] (Physical Resource, Logical Resource, Managed O-Cloud Service). The bridge does not extend the spec; it maps operator-meaningful action classes (host configuration, driver swap, node firmware, performance profile, SR-IOV allocation, and twelve others) onto pre-existing O-RAN class boundaries.

3.3 Bridge 3: O-RAN O2 IMS to Kubernetes-native CRDs

This is the architectural core of the harness. The routing rule at `harness/routing-rules/contribution-1-routing-rule.yaml` dispatches infrastructure-layer faults through the O2 IMS provisioning surface [23, 24] into Kubernetes-native CRD reconcilers. The reference O2 IMS implementation is Red Hat’s O-Cloud Manager [26]. The bridge produces what the source repository calls the 2+3+8+9 conjunction (numbers from the AMA-numbered bibliography in `docs/references.md`, not from this paper’s rendered bibliography): a single `RemediationProposal` travels through the O-RAN-standardized provisioning request envelope into either Metal3 HostFirmware-Components [14] or the Machine Config Operator [27]. Kernel Module Management v1beta1 [8] is a co-delivery path within the same bridge for driver-swap action classes; the four-element conjunction names the two CRDs that anchor the novelty claim against publicly-published architectures, with KMM as a third concrete reconciler that the same routing surface dispatches into.

3.4 Bridge 4: O-RAN routing decision to TM Forum intent (TMF921)

Service-layer faults route up via TMF921 [36]. The harness emits a companion intent and observes the partner SMO’s response. The dual-route pattern also fires this bridge for infrastructure-layer faults whose blast radius implies a service window (the canonical case is a node firmware update that forces a host reboot). The bridge crosses the O-RAN resource-layer classification into the TM Forum intent envelope; the audit emits both legs.

3.5 Bridge 5: DMTF Redfish to Metal3 to O-RAN O2 IMS

When a firmware push targets a host BMC, Metal3 BMO [14] issues a Redfish `UpdateService.SimpleUpdate` call [4]. The harness does not author a Redfish binding; it delegates to Metal3, which in turn responds to the O2 IMS reconciler invocation. The bridge spans three bodies (O-RAN, CNCF / Kubernetes, DMTF) in one delivery path. The honest-scope acknowledgement here: the harness in v0 references the Redfish path through Metal3 stubs but does not exercise it against live BMCs.

3.6 Bridge 6: TM Forum TMF688 as the audit sink

Every `RemediationProposal` exits as a TMF688-shaped `AuditEvent` [35]. The audit carries the proposal, the guardrail result, the sandbox verdict, the O2 IMS dispatch envelope (if the routing target was infrastructure), the companion intent and its dispatch result (if the routing emitted a TMF921 companion), and a seven-field `ReversibilityProfile`. The bridge here is: an O-RAN-classified action plus a Kubernetes-native

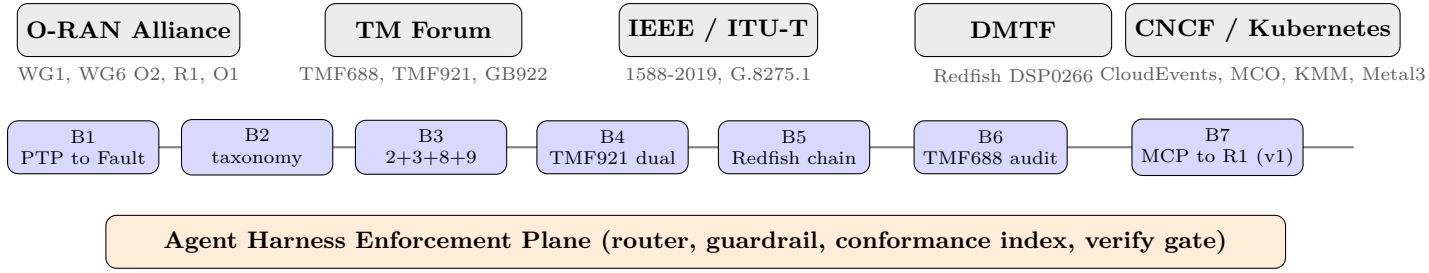


Figure 1: Seven contract bridges between five standards bodies and the harness enforcement plane. Each numbered bridge corresponds to a subsection of this section. Specific interface references appear under each standards body box. The enforcement plane below the rail is where each bridge becomes machine-verifiable via the citation header on the relevant authored YAML or JSON artifact.

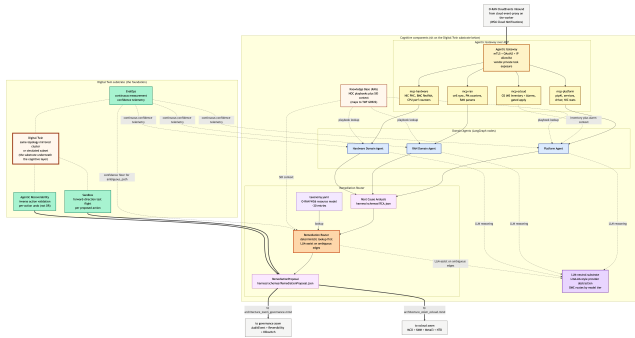


Figure 2: Cognitive middle zoom. The Agentic Gateway terminates the CloudEvents stream and presents the MCP server mesh (mcp-platform, mcp-ran, mcp-hardware, mcp-ocloud) to the three domain agents (Platform, RAN, Hardware). The deterministic Router consults the taxonomy and the routing rule. EvalOps, Sandbox, and Agentic Recoverability run as activities on the Digital Twin substrate beneath. Citation-header enforcement and the conformance index sit alongside the Router as machine-verifiable contracts.

payload plus an O-RAN-cited delivery dispatch land inside a TM Forum event envelope as the operator-facing record. TMF688 is the cross-spec lingua franca for the audit row.

3.7 Bridge 7: MCP internal scaffolding to O-RAN R1 (prospective)

The harness’s Agentic Gateway exposes vendor-private tooling to agents via MCP [1, 2, 13]. MCP is NOT an O-RAN management plane interface; the layering is declared explicitly in the harness’s `contribution-3-llm-neutrality.yaml`. A future rApp realization that exposes harness capability to other SMO components would expose via R1 [25]. The bridge is prospective: the seam is declared in the disclaimer and a v1 rApp deliverable can wire it up without disturbing the v0 internal scaffolding.

4 The Conformance Auditing Mechanism

The bridge claims are only useful if a reviewer can confirm them mechanically. The harness ships two enforcement mechanisms.

Citation headers on every authored stub. Every YAML and JSON file under `harness/` and `scenarios/` opens with a `_conforms_to` or equivalent block naming the spec, the spec section, and the bibliography references. Failure to add a citation header on a new file fails the verify gate’s first check. The mechanism turns documentation into a contract.

Bidirectional conformance index. A file at `harness/conformance.md` carries a two-column table mapping every authored artifact to its spec lineage. The verify gate’s fourth check confirms both directions: no file in the harness tree is missing from the index, and no row in the index points to a file that does not exist. The mechanism prevents the index from rotting silently as the codebase changes.

A reviewer who disputes any bridge claim in Section 3 runs `python3 scripts/verify.py` from the repository root. Check 1 confirms that every cited file has a citation header. Check 4 confirms that the index is bidirectionally complete. Check 9 (when `jsonschema` is installed) confirms scenario artifacts validate against the JSON Schemas. Check 10 replays the walker against every committed scenario fixture and compares output on seven material fields. The gate exits non-zero on any failure with a specific row identifying the broken bridge.

5 Comparison to Adjacent Projects

ONAP [10]. The Open Network Automation Platform tends to absorb adjacent specs into the ONAP information model and policy framework. This is operationally successful at scale but produces a centralizing posture: ONAP is the authoritative frame, and other specs are mapped into it. The harness pattern described here takes the opposite posture: each spec retains domain authority and the harness composes at named seams. A deployment that runs ONAP for orchestration could compose with this closed-loop harness at the audit

and intent bridges (Bridges 4 and 6) without rewriting ONAP's internal model.

Nephio [9]. Nephio is cloud-native network automation focused on KRM-based package specialization for orchestration. It does not ship a closed-loop remediation harness. The composition is natural: Nephio delivers the standing topology and post-deployment configuration; the harness handles Day-2 anomalies on top of that topology. Bridges 2, 3, and 6 do not conflict with Nephio's package model; they consume the cluster state Nephio produces.

Open Source MANO [11]. OSM is mature management and orchestration software. It is spec-aligned with the broader telco automation stack but does not ship a closed-loop remediation substrate with a deterministic taxonomy grounded in O-RAN's resource model. The harness can sit beside OSM in a deployment that wants to use OSM for VNF lifecycle and the harness for Cloud RAN infrastructure remediation.

Honest scope. The composition claim is bridge-by-bridge, not interop-tested. Several bridges (notably Bridge 5 for live Redfish writes and Bridge 7 for an actual R1 exposure) are present at the schema or disclaimer level without live interop. The path from "v0 bridges are documented and locally verified" to "v1 bridges are interop-tested against vendor implementations" is future work. The standards-track ask is twofold: an O-RAN nGRG output that names a Closed-Loop Remediation SMOS as a first-class element of the Decoupled SMO catalogue, and a TMF Implementation Guide that documents the harness-unique dispatch result extension and the seven-field ReversibilityProfile as candidate API additions.

6 Conclusion

This paper has documented a bridge-based composition architecture that lets a closed-loop Cloud RAN remediation harness touch contracts from five standards bodies (O-RAN Alliance, TM Forum, IEEE, DMTF, CNCF / Kubernetes SIGs) without authoring a single parallel specification. Seven named bridges are documented with file-level citation anchors. Two enforcement mechanisms (citation headers and a bidirectional conformance index) make the bridges machine-verifiable. The verify gate runs in five seconds and exits binary pass/fail. We argued that the resulting architecture is more durable across spec revisions and politically simpler to land in a multivendor ecosystem than approaches that elevate one body or invent a parallel spec. The v0 limits are declared and the v1 path is named.

Acknowledgements

The architecture extends a prior multivendor diagnostic substrate co-authored with S. Kiani Mehr, N. Korati Prasanna, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh [7]. The closed-loop extension and the bridge architecture documented in this paper are open-source

under Apache-2.0 at <https://github.com/dkypuros/oran-agent-harness>.

References

- [1] Anthropic. Model context protocol python SDK. <https://github.com/modelcontextprotocol/python-sdk>, 2026. Accessed 14 May 2026.
- [2] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2026. Accessed 14 May 2026.
- [3] Cloud Native Computing Foundation. CloudEvents version 1.0 specification: a vendor-neutral specification for describing event data. <https://cloudevents.io>, 2026. Accessed 14 May 2026.
- [4] Distributed Management Task Force. Redfish specification (DMTF DSP0266). <https://www.dmtf.org/standards/redfish>, 2026. Out-of-band management of servers and infrastructure. Accessed 14 May 2026.
- [5] Institute of Electrical and Electronics Engineers. IEEE Std 1588-2019: IEEE standard for a precision clock synchronization protocol for networked measurement and control systems. <https://standards.ieee.org/ieee/1588/6825/>, 2019.
- [6] International Telecommunication Union, Telecommunication Standardization Sector. ITU-T G.8275.1: Precision time protocol telecom profile for phase / time synchronization with full timing support from the network. <https://www.itu.int/rec/T-REC-G.8275.1>, 2022.
- [7] S. Kiani Mehr, N. Korati Prasanna, D. Kypuros, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh. Multivendor agentic AI solution for cloud RAN troubleshooting. Ericsson Technology Blog. <https://www.ericsson.com/en/blog/2026/5/multivendor-agentic-ai-solution-for-cloud-ran>, 2026. Published 11 May 2026. Accessed 14 May 2026.
- [8] Kubernetes Special Interest Groups. Kernel module management operator (KMM). <https://github.com/kubernetes-sigs/kernel-module-management>, 2026. Accessed 14 May 2026.
- [9] Linux Foundation Networking. Nephio: Cloud-native network automation. <https://nephio.org>, 2026. Accessed 14 May 2026.
- [10] Linux Foundation Networking. Open network automation platform (ONAP). <https://www.onap.org>, 2026. Accessed 14 May 2026.
- [11] Linux Foundation Networking. Open source MANO (OSM). <https://osm.etsi.org>, 2026. Accessed 14 May 2026.

- [12] linuxptp project. ptp4l(8) manual page: PTP boundary / ordinary clock daemon for Linux. <https://linuxptp.sourceforge.net>, 2026. Accessed 14 May 2026.
- [13] J. Lowin. FastMCP: a pythonic framework for building model context protocol servers and clients. <https://github.com/jlowin/fastmcp>, 2026. Accessed 14 May 2026.
- [14] Metal3 project. Baremetal operator: BareMetal-Host, HostFirmwareSettings, HostFirmwareComponents Custom Resource Definitions. <https://github.com/metal3-io/baremetal-operator>, 2026. Accessed 14 May 2026.
- [15] Metal3 project. Metal3: Bare metal host provisioning for Kubernetes. <https://metal3.io>, 2026. Accessed 14 May 2026.
- [16] O-RAN Alliance. O-RAN A1 interface: General aspects and principles and application protocol, working group 2 (O-RAN.WG2.TS.A1GAP-R005-v05.03 and O-RAN.WG2.TS.A1AP-R005-v06.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [17] O-RAN Alliance. O-RAN architecture description, working group 1 specification. <https://www.o-ran.org/specifications>, 2026. Accessed 14 May 2026.
- [18] O-RAN Alliance. O-RAN cloud notifications specification, working group 6 (O-RAN.WG6.Cloud-Notifications). <https://www.o-ran.org/specifications>, 2026. Accessed 14 May 2026.
- [19] O-RAN Alliance. O-RAN decoupled SMO architecture, working group 1 technical report (O-RAN.WG1.TR.Decoupled-SMO-Architecture-R004-v03.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026. Source for SMOS terminology.
- [20] O-RAN Alliance. O-RAN E2 interface family: General aspects and principles, application protocol, and service models, working group 3 (E2GAP, E2AP, E2SM-KPM, E2SM-RC, E2SM-LLC). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [21] O-RAN Alliance. O-RAN O1 interface specification, working group 10 (O-RAN.WG10.TS.O1-Interface.0-R005-v18.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [22] O-RAN Alliance. O-RAN O2 DMS interface specification (deployment management services), O-RAN.WG6.O2DMS-Interface. <https://www.o-ran.org/specifications>, 2026. Working Group 6 specification. Accessed 14 May 2026.
- [23] O-RAN Alliance. O-RAN O2 general aspects and principles, version 01.02 (O-RAN.WG6.O2-GANP-v01.02). <https://www.o-ran.org/specifications>, 2026. Working Group 6 specification, with successor in R005 release train (O-RAN.WG6.TS.O2-GA&P-R005-v10.00). Accessed 14 May 2026.
- [24] O-RAN Alliance. O-RAN O2 IMS interface specification (infrastructure management services), O-RAN.WG6.O2IMS-Interface. <https://www.o-ran.org/specifications>, 2026. Working Group 6 specification. Accessed 14 May 2026.
- [25] O-RAN Alliance. O-RAN R1 interface: General aspects and principles and application protocols for R1 services, working group 2 (O-RAN.WG2.TS.R1GAP-R005-v13.00 and O-RAN.WG2.TS.R1AP-R005-v10.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [26] OpenShift KNI. oran-o2ims: O-Cloud manager. <https://github.com/openshift-kni/oran-o2ims>, 2026. Accessed 14 May 2026.
- [27] OpenShift project. Machine config operator: declarative configuration of Red Hat Enterprise Linux CoreOS. <https://github.com/openshift/machine-config-operator>, 2026. Accessed 14 May 2026.
- [28] Red Hat Cloud Native Events project. cloud-event-proxy: PTP notification publisher emitting O-RAN-compliant CloudEvents from linuxptp state changes. <https://github.com/redhat-cne/cloud-event-proxy>, 2026. Accessed 14 May 2026.
- [29] TM Forum. GB922 information framework (SID) concepts and principles. <https://www.tmforum.org>, 2026. Accessed 14 May 2026.
- [30] TM Forum. IG1190 open digital architecture functional architecture. <https://www.tmforum.org>, 2026. Accessed 14 May 2026.
- [31] TM Forum. IG1228 AIOps and intelligence management domain implementation guide. <https://www.tmforum.org>, 2026. Accessed 14 May 2026.
- [32] TM Forum. IG1253 / IG1253D intent manager implementation guide. <https://www.tmforum.org>, 2026. Accessed 14 May 2026.
- [33] TM Forum. TMF634 resource inventory management API specification. <https://www.tmforum.org/oda/open-apis/table/tmf634>, 2026. Accessed 14 May 2026.
- [34] TM Forum. TMF642 alarm management API specification. <https://www.tmforum.org/oda/open-apis/table/tmf642>, 2026. Accessed 14 May 2026.

- [35] TM Forum. TMF688 event management API user guide and REST specification. <https://www.tmforum.org/oda/open-apis/table/tmf688>, 2026. Accessed 14 May 2026.
- [36] TM Forum. TMF921 intent management API user guide and REST specification. <https://www.tmforum.org/oda/open-apis/table/tmf921>, 2026. Accessed 14 May 2026.